LAPORAN PRAKTEK

PRAKTEK SISTEM BASIS DATA NOSQL

MUHAMMAD IZZUDIN IRSYAD (225443018)

1AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

Politeknik Manufaktur Bandung

Tahun Ajaran 2025/2026

PENDAHULUAN

Dalam proses pengolahan data, penggunaan database sangat penting untuk menyimpan dan menganalisis data secara efisien. Salah satu database yang sering digunakan adalah MongoDB, yang memiliki fitur aggregation pipeline untuk melakukan analisis data secara terstruktur.

Pada praktikum ini dilakukan pengolahan data sensor dengan menggunakan beberapa pipeline, seperti filtering data, perhitungan rata-rata, analisis nilai ekstrem, serta pengelompokan berdasarkan waktu. Selain itu, juga dilakukan proses penggabungan data antar koleksi menggunakan $lookup agar informasi yang dihasilkan menjadi lebih lengkap dan mudah dipahami.

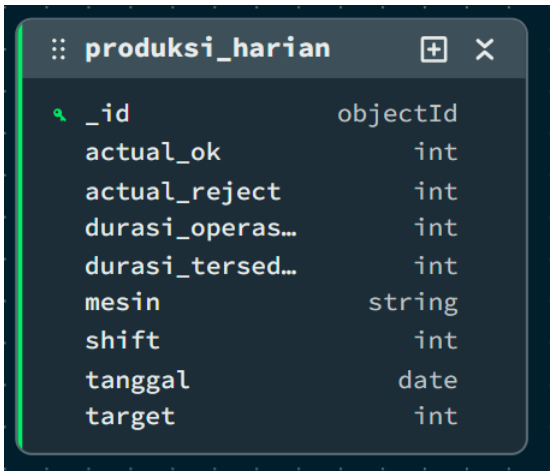
Tujuan dari praktikum ini adalah untuk memahami cara kerja aggregation pipeline serta mengimplementasikannya dalam menganalisis data sensor sehingga dapat menghasilkan informasi yang berguna.

ISI

PRAKTIK IMPLEMENTASI AGGREGATION FRAMEWORK

A. TUTORIAL TERBIMBING STUDI KASUS AGREGASI

Desain Data:



1. Membuat Database

   use pabrik_elektronik

   Output:

   ```
   switched to db pabrik_elektronik
   ```

2. Persiapan Data

   a. Membuat Validasi Data

   ```
   db.createCollection("produksi_harian", {
     validator: {
       $jsonSchema: {
         bsonType: "object",
         required: ["mesin", "tanggal", "shift", "target", "actual_ok", "actual_reject", "durasi_operasi_menit", "durasi_tersedia_menit"],
         properties: {
           mesin: {
             bsonType: "string",
             description: "Kode mesin (M01 – M10)"
           },
           tanggal: {
             bsonType: "date",
             description: "Tanggal produksi"
           },
           shift: {
             bsonType: "int",
             description: "Shift (1,2,3)"
           },
   ```

```
                                    target: {
          bsonType: "int",
          description: "Target produksi (unit)"
        },
        actual_ok: {
          bsonType: "int",
          description: "Hasil baik aktual"
        },
        actual_reject: {
          bsonType: "int",
          description: "Hasil cacat"
        },
        durasi_operasi_menit: {
          bsonType: "int",
          description: "Waktu operasi aktual (menit)"
        },
        durasi_tersedia_menit: {
          bsonType: "int",
          description: "Waktu tersedia (menit)"
        }
      }
    }
  }
})
```

Output:

```
< { ok: 1 }
```

Penjelasan:

Membuat Validasi dokumen, sehingga data jadi konsisten, menghindari error saat perhitungan

b. Membuat Data Dummy

```
db.produksi_harian.drop()
var docs = [];
var mesinList = []; for (let i=1;i<=10;i++) mesinList.push("M"+i.toString().padStart(2,'0'));
var start = new Date(2026,3,1); // April
for (let i=0; i<1000; i++) {
  var tgl = new Date(start.getTime() + Math.floor(Math.random()*90)*86400000);
  var shift = Math.floor(Math.random()*3)+1;
  var target = Math.floor(Math.random()*301)+200;
  var actual_ok = Math.floor(target * (0.6 + Math.random()*0.4));
  var actual_reject = Math.floor(actual_ok * Math.random()*0.15);
  var durasi_tersedia = 480;
  var durasi_operasi = Math.floor(durasi_tersedia * (0.7 + Math.random()*0.3));
  docs.push({
    mesin: mesinList[Math.floor(Math.random()*10)],
    tanggal: tgl,
```

```
        shift: shift,
        target: target,
        actual_ok: actual_ok,
        actual_reject: actual_reject,
        durasi_operasi_menit: durasi_operasi,
        durasi_tersedia_menit: durasi_tersedia
      });
      if (docs.length === 500) { db.produksi_harian.insertMany(docs); docs = []; }
    }
    if (docs.length) db.produksi_harian.insertMany(docs);
```

Output:

```
< []
```

Penjelasan:
Tujuan dari codingan ini untuk membuat data simulasi/dummy

c. Verifikasi Data

```
db.produksi_harian.countDocuments()
db.produksi_harian.findOne()
```

Output:

```
{
  _id: ObjectId('69f81013a8384375cdd4b163'),
  mesin: 'M05',
  tanggal: 2026-05-22T17:00:00.000Z,
  shift: 1,
  target: 484,
  actual_ok: 409,
  actual_reject: 46,
  durasi_operasi_menit: 408,
  durasi_tersedia_menit: 480
}
```

Penjelasan:
Memverifikasi apakah data dummy sudah tersedia.

3. Menghitung OEE Per Mesin Per Bulan

```
db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan
  { $group: {
      _id: {
        mesin: "$mesin",
        bulan: { $month: "$tanggal" },
        tahun: { $year: "$tanggal" }
      },
      total_target: { $sum: "$target" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_op: { $sum: "$durasi_operasi_menit" },
      total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
      _id: 0,
```

```
      mesin: "$_id.mesin",
      bulan: "$_id.bulan",
      tahun: "$_id.tahun",
      total_ok: 1,
      total_target: 1,
      total_reject: 1,
      availability: { $divide: ["$total_op", "$total_avail"] },
      performance: { $divide: ["$total_ok", "$total_target"] },
      quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
      OEE: {
        $multiply: [
          { $divide: ["$total_ok", "$total_target"] },
          { $divide: ["$total_op", "$total_avail"] },
          { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
        ]
      }
    }},
    // Tahap 4: Urutkan berdasarkan OEE terendah
    { $sort: { OEE: 1 } }
  ])
```

Output:


```
< {
    total_target: 859,
    total_ok: 561,
    total_reject: 34,
    mesin: 'M07',
    bulan: 3,
    tahun: 2026,
    availability: 0.8885416666666667,
    performance: 0.6530849825378346,
    quality: 0.9428571428571428,
    OEE: 0.5471336063529021
  }
  {
    total_target: 10198,
    total_ok: 7792,
    total_reject: 725,
    mesin: 'M03',
    bulan: 4,
    tahun: 2026,
    availability: 0.8320833333333333,
    performance: 0.7640713865463816,
    quality: 0.9148761300927557,
    OEE: 0.5816517726902519
  }
  {
    total_target: 495,
    total_ok: 420,
    total_reject: 42,
    mesin: 'M06',
    bulan: 3,
    tahun: 2026,
    availability: 0.7666666666666667,
    performance: 0.8484848484848485,
    quality: 0.9090909090909091,
    OEE: 0.5913682277318641
```


Penjelasan:

Menghitung performa mesin agar mengetahui mesin mana yang baik maupun buruk

4. Menemukan Mesin dengan OEE di Bawah 80%

```
db.produksi_harian.aggregate([
  { $group: {
      _id: {
        mesin: "$mesin",
        bulan: { $month: "$tanggal" },
        tahun: { $year: "$tanggal" }
      },
      total_target: { $sum: "$target" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_op: { $sum: "$durasi_operasi_menit" },
      total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
      _id: 0,
      mesin: "$_id.mesin",
      bulan: "$_id.bulan",
      tahun: "$_id.tahun",
      total_ok: 1,
      total_target: 1,
      total_reject: 1,
      availability: { $divide: ["$total_op", "$total_avail"] },
      performance: { $divide: ["$total_ok", "$total_target"] },
      quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
      OEE: {
        $multiply: [
          { $divide: ["$total_ok", "$total_target"] },
          { $divide: ["$total_op", "$total_avail"] },
          { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
        ]
      }
  }},
  { $match: { OEE: { $lt: 0.80 } } },
  { $sort: { OEE: 1 } }
])
```

Output:

```
< {
    total_target: 859,
    total_ok: 561,
    total_reject: 34,
    mesin: 'M07',
    bulan: 3,
    tahun: 2026,
    availability: 0.8885416666666667,
    performance: 0.6530849825378346,
    quality: 0.9428571428571428,
    OEE: 0.5471336063529021
  }
  {
    total_target: 10198,
    total_ok: 7792,
    total_reject: 725,
    mesin: 'M03',
    bulan: 4,
    tahun: 2026,
    availability: 0.8320833333333333,
    performance: 0.7640713865463816,
    quality: 0.9148761300927557,
    OEE: 0.5816517726902519
  }
  {
    total_target: 495,
    total_ok: 420,
    total_reject: 42,
    mesin: 'M06',
    bulan: 3,
    tahun: 2026,
    availability: 0.7666666666666667,
    performance: 0.8484848484848485,
    quality: 0.9090909090909091,
    OEE: 0.5913682277318641
```

Penjelasan:
Menghitung dan mencari performa mesin bermasalah atau di bawah 80%

5. Distribusi Jumlah Produksi Per Shift Dalam Bucket
   a. Distribusi sebelum dipisah per-shift

```
db.produksi_harian.aggregate([
  { $bucket: {
      groupBy: "$actual_ok",
      boundaries: [0, 200, 300, 400, 600], // 0-199, 200-299, 300-399, 400-600
      default: "di atas 600",
      output: {
        count: { $sum: 1 },
        rata_reject: { $avg: "$actual_reject" }
      }
  }}
])
```

Output:

```
< {
    _id: 0,
    count: 173,
    rata_reject: 12.22543352601156
  }
  {
    _id: 200,
    count: 423,
    rata_reject: 18.79432624113475
  }
  {
    _id: 300,
    count: 303,
    rata_reject: 26.392739273927393
  }
  {
    _id: 400,
    count: 101,
    rata_reject: 30.257425742574256
  }
```

Penjelasan:
Mengelompokkan jumlah produksi actual_ok distribusi produksi secara umum

b. Distribusi Dipisah Per-shift

```
db.produksi_harian.aggregate([
  { $group: {
      _id: "$shift",
      data: { $push: "$actual_ok" }  // kumpulkan nilai
  }},
  // Kemudian di aplikasi atau gunakan operator $bucket di sub-pipeline (tidak bisa langsung karena $bucket butuh array dokumen)
])
```

Output:

```
< {
    _id: 2,
    data: [
      206,
      269,
      470,
      274,
      338,
      424,
      290,
      393,
      213,
      135,
      291,
      403,
```

Penjelasan:
Mengumpulkan data Per-shift, tetapi belum bisa dibucket dan perlu diproses lebih lanjut

c. Alternatif: Gunakan $facet untuk menjalankan pipeline bucket paralel per shift

```
db.produksi_harian.aggregate([
```

```
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count: {$sum:1} } } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count: {$sum:1} } } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count: {$sum:1} } } }
    ]
  }}
])
```

Output:

```
< {
    shift1: [
      {
        _id: 0,
        count: 58
      },
      {
        _id: 200,
        count: 127
      },
      {
        _id: 300,
        count: 111
      },
      {
        _id: 400,
        count: 34
      }
    ],
    shift2: [
      {
        _id: 0,
        count: 57
      },
      {
        _id: 200,
        count: 154
      },
      {
        _id: 300,
        count: 99
      },
      {
        _id: 400,
        count: 31
```

Penjelasan:
Distribusi produksi per-shift langsung

d. Cara sederhana Jumlah Per-shift

```
db.produksi_harian.aggregate([
  { $group: { _id: "$shift", total_produksi: { $sum: "$actual_ok" } } }
])
```

Output:

```
< {
    _id: 2,
    total_produksi: 95476
  }
  {
    _id: 1,
    total_produksi: 94599
  }
  {
    _id: 3,
    total_produksi: 92432
  }
```

Penjelasan:
Menggunakan cara sederhana dengan total produksi tiap shift

6. Optimasi dan Analisis
   db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
   Output:

```
< mesin_1_tanggal_1
```

   Penjelasan:
   Mempercepat Query menganalisis permesin dan waktu

7. Latihan Tambahan
   a. Tren Bulanan: Modifikasi pipeline agar menampilkan OEE per bulan untuk semua mesin, lalu urutkan bulan.

```
db.produksi_harian.aggregate([
  {
   $group: {
     _id: {
       bulan: { $month: "$tanggal" },
       tahun: { $year: "$tanggal" }
     },
     total_ok: { $sum: "$actual_ok" },
     total_reject: { $sum: "$actual_reject" },
     total_target: { $sum: "$target" },
     total_op: { $sum: "$durasi_operasi_menit" },
     total_avail: { $sum: "$durasi_tersedia_menit" }
   }
  },
  {
   $project: {
     _id: 0,
     bulan: "$_id.bulan",
     tahun: "$_id.tahun",
     OEE: {
```

```
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        {
          $divide: [
            "$total_ok",
            { $add: ["$total_ok", "$total_reject"] }
          ]
        }
      ]
    }
  }
},
{ $sort: { tahun: 1, bulan: 1 } }
])
```

Output:

```
< {
    bulan: 3,
    tahun: 2026,
    OEE: 0.6717383675232707
  }
  {
    bulan: 4,
    tahun: 2026,
    OEE: 0.630320648132556
  }
  {
    bulan: 5,
    tahun: 2026,
    OEE: 0.6295097015625095
  }
  {
    bulan: 6,
    tahun: 2026,
    OEE: 0.6467065117321874
  }
```


Penjelasan:

- $group Menggabungkan semua data berdasarkan bulan & tahun, lalu menjumlahkan total produksi dan durasi
- $project Menghitung nilai OEE dari total yang sudah dijumlahkan
- $sort Mengurutkan data berdasarkan waktu (tahun, bulan) agar terlihat tren

b. Mesin Terburuk: Tampilkan 3 mesin dengan OEE terendah di bulan Juni 2026 saja (gunakan $match di awal untuk filter bulan).

```
db.produksi_harian.aggregate([
  {
    $match: {
      tanggal: {
        $gte: new Date("2026-06-01"),
        $lt: new Date("2026-07-01")
      }
    }
  },
```

```
  {
   $group: {
     _id: "$mesin",
     total_ok: { $sum: "$actual_ok" },
     total_reject: { $sum: "$actual_reject" },
     total_target: { $sum: "$target" },
     total_op: { $sum: "$durasi_operasi_menit" },
     total_avail: { $sum: "$durasi_tersedia_menit" }
   }
  },
  {
   $project: {
     mesin: "$_id",
     OEE: {
       $multiply: [
         { $divide: ["$total_ok", "$total_target"] },
         { $divide: ["$total_op", "$total_avail"] },
         {
           $divide: [
             "$total_ok",
             { $add: ["$total_ok", "$total_reject"] }
           ]
         }
       ]
     }
   }
  },
  { $sort: { OEE: 1 } },
  { $limit: 3 }
])
```

Output:

```
< {
    _id: 'M01',
    mesin: 'M01',
    OEE: 0.6252891106004828
  }
  {
    _id: 'M05',
    mesin: 'M05',
    OEE: 0.6327890743182639
  }
  {
    _id: 'M06',
    mesin: 'M06',
    OEE: 0.6344009303535765
  }
```

Penjelasan:

- $match Memfilter data hanya untuk bulan Juni 2026$group Mengelompokkan data per mesin dan menjumlahkan semua parameter

- $group Mengelompokkan data per mesin dan menjumlahkan semua parameter
- $project Menghitung nilai OEE tiap mesin
- $sort Mengurutkan dari OEE terendah
- $limit Mengambil hanya 3 mesin terburuk

c. Kinerja Harian: Hitung OEE per mesin per hari (gunakan $dateToString untuk ekstrak tanggal).

```
db.produksi_harian.aggregate([
  {
   $group: {
     _id: {
       mesin: "$mesin",
       tanggal: {
         $dateToString: { format: "%Y-%m-%d", date: "$tanggal" }
       }
     },
     total_ok: { $sum: "$actual_ok" },
     total_reject: { $sum: "$actual_reject" },
     total_target: { $sum: "$target" },
     total_op: { $sum: "$durasi_operasi_menit" },
     total_avail: { $sum: "$durasi_tersedia_menit" }
   }
  },
  {
   $project: {
     _id: 0,
     mesin: "$_id.mesin",
     tanggal: "$_id.tanggal",
     OEE: {
       $multiply: [
         { $divide: ["$total_ok", "$total_target"] },
         { $divide: ["$total_op", "$total_avail"] },
         {
           $divide: [
             "$total_ok",
             { $add: ["$total_ok", "$total_reject"] }
           ]
         }
       ]
     }
   }
  },
  { $sort: { tanggal: 1 } }
])
```

Output:

```
< {
    mesin: 'M05',
    tanggal: '2026-03-31',
    OEE: 0.8092093347265761
  }
  {
    mesin: 'M09',
    tanggal: '2026-03-31',
    OEE: 0.6183754321181647
  }
  {
    mesin: 'M01',
    tanggal: '2026-03-31',
    OEE: 0.8120212884737057
  }
  {
    mesin: 'M02',
    tanggal: '2026-03-31',
    OEE: 0.647461466802168
  }
  {
    mesin: 'M10',
    tanggal: '2026-03-31',
    OEE: 0.7305420266672414
  }
  {
    mesin: 'M07',
    tanggal: '2026-03-31',
    OEE: 0.5471336063529021
  }
  {
```

Penjelasan:

- $group Mengelompokkan data berdasarkan mesin dan tanggal (diubah ke format string)
- $project Menghitung nilai OEE harian
- $sort Mengurutkan berdasarkan tanggal agar kronologis

d. Simulasi Perbaikan: Jika quality harus minimal 95%, tampilkan mesin yang tidak memenuhi kriteria tersebut.

```
db.produksi_harian.aggregate([
 {
  $group: {
    _id: "$mesin",
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" }
  }
},
{
  $project: {
    mesin: "$_id",
    quality: {
      $divide: [
        "$total_ok",
```

```
        { $add: ["$total_ok", "$total_reject"] }
      ]
    }
  }
},
{
  $match: {
    quality: { $lt: 0.95 }
  }
},
{ $sort: { quality: 1 } }
])
```

Output:

```
< {
    mesin: 'M05',
    tanggal: '2026-03-31',
    OEE: 0.8092093347265761
  }
  {
    mesin: 'M09',
    tanggal: '2026-03-31',
    OEE: 0.6183754321181647
  }
  {
    mesin: 'M01',
    tanggal: '2026-03-31',
    OEE: 0.8120212884737057
  }
  {
    mesin: 'M02',
    tanggal: '2026-03-31',
    OEE: 0.647461466802168
  }
  {
    mesin: 'M10',
    tanggal: '2026-03-31',
    OEE: 0.7305420266672414
  }
  {
    mesin: 'M07',
    tanggal: '2026-03-31',
    OEE: 0.5471336063529021
  }
  {
```

Penjelasan:

- $group Menjumlahkan total produksi OK dan reject per mesin
- $project Menghitung nilai quality (rasio produk baik)
- $match Memfilter mesin dengan quality < 95%
- $sort Mengurutkan dari kualitas terendah

ANALISIS DAN KESIMPULAN

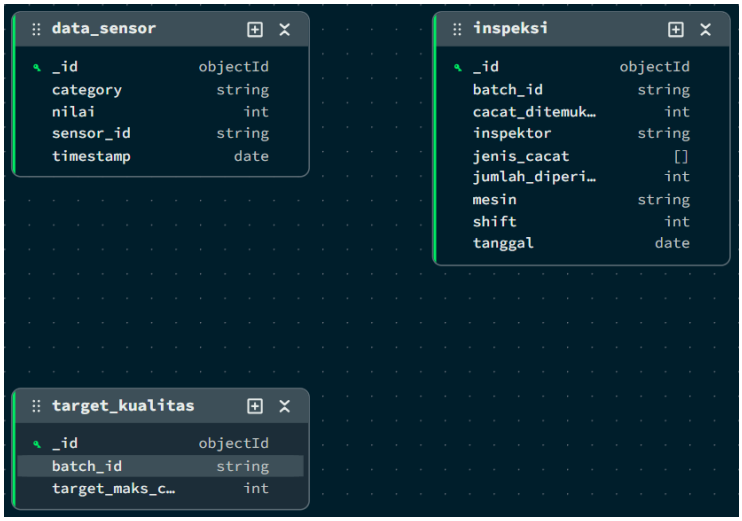
Berdasarkan hasil pengolahan data menggunakan aggregation pipeline di MongoDB, terlihat bahwa seluruh mesin yang ditampilkan memiliki nilai OEE di bawah 80%, dengan rentang sekitar 0.54 sampai 0.63. Hal ini menunjukkan bahwa performa produksi secara umum masih belum optimal dan perlu dilakukan perbaikan.

Dari data tersebut, mesin dengan performa paling rendah adalah M07 pada bulan Maret 2026 dengan nilai OEE sekitar 0.547. Jika dilihat lebih detail, penyebab utamanya adalah nilai performance yang rendah (sekitar 0.65), meskipun availability dan quality sudah cukup baik. Hal ini menunjukkan bahwa mesin belum mampu mencapai target produksi secara maksimal.

Dari tren yang terlihat, sebagian besar mesin memiliki nilai availability di atas 0.8 dan quality mendekati atau di atas 0.9. Artinya, masalah utama bukan pada downtime atau kualitas, tetapi lebih ke ketidakmampuan mencapai target produksi (performance).

B. Panduan Mandiri

Desain Data:



1. Membuat Database dan Membuat Data Dummy

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

# Ganti dengan NIM Anda
NIM = "2206054321"
client = MongoClient('mongodb://localhost:27017')
db = client[f'tugas5_{NIM}']
db.inspeksi.drop()
db.target_kualitas.drop()

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
batch_ids = [f"B{str(i).zfill(4)}" for i in range(1, 2001)]
inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"]

# Generate data inspeksi (1500 dokumen)
docs = []
start_date = datetime(2026, 4, 1, 6, 0)
```

```
for i in range(1500):
    batch = random.choice(batch_ids)
    mesin = random.choice(mesin_list)
    tanggal = start_date + timedelta(minutes=random.randint(0, 60*24*90))  # 3 bulan
    shift = 1 if tanggal.hour < 14 else (2 if tanggal.hour < 22 else 3)
    jumlah = random.randint(100, 500)
    reject = int(jumlah * random.uniform(0, 0.10))  # 0-10% cacat
    jenis = random.sample(jenis_cacat_options, k=random.randint(1, 3))
    doc = {
        "batch_id": batch,
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "inspektor": random.choice(inspektor_list),
        "jumlah_diperiksa": jumlah,
        "cacat_ditemukan": reject,
        "jenis_cacat": jenis
    }
    docs.append(doc)
    if len(docs) >= 500:
        db.inspeksi.insert_many(docs)
        docs.clear()
if docs:
    db.inspeksi.insert_many(docs)

print("Data inspeksi selesai. Total:", db.inspeksi.count_documents({}))

# Generate target kualitas (setiap batch punya target cacat maksimal 5% dari rata-rata ukuran
batch)
targets = []
for i in range(1, 2001):
    batch = f"B{str(i).zfill(4)}"
    # target cacat acak 1-15
    target_cacat = random.randint(1, 15)
    targets.append({"batch_id": batch, "target_maks_cacat": target_cacat})
    if len(targets) >= 500:
        db.target_kualitas.insert_many(targets)
        targets.clear()
if targets:
    db.target_kualitas.insert_many(targets)

print("Data target kualitas selesai. Total:", db.target_kualitas.count_documents({}))
```

Output:

```
Data inspeksi selesai. Total: 1500
Data target kualitas selesai. Total: 2000
```

Penjelasan:

2. Verifikasi Data
   a. db.inspeksi.countDocuments() // harus >= 1000
      Output:

```
< 1500
```

   b. db.target_kualitas.countDocuments() // harus >= 50
      Output:

```
< 2000
```

   c. db.inspeksi.findOne()
      Output:

```
< {
    _id: ObjectId('69f84cd5e371fd1bacdc5dc5'),
    batch_id: 'B0693',
    mesin: 'M07',
    tanggal: 2026-05-09T21:28:00.000Z,
    shift: 2,
    inspektor: 'Dian',
    jumlah_diperiksa: 450,
    cacat_ditemukan: 24,
    jenis_cacat: [
      'pori',
      'gores',
      'bengkok'
    ]
  }
```

   d. db.target_kualitas.findOne()
      Output:

```
< {
    _id: ObjectId('69f84cd5e371fd1bacdc63a1'),
    batch_id: 'B0001',
    target_maks_cacat: 13
  }
```

   e. db.inspeksi.createIndex({ batch_id: 1 })
      Output:

```
< batch_id_1
```

   f. db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })
      Output:

```
< mesin_1_tanggal_1
```

   g. db.target_kualitas.createIndex({ batch_id: 1 })
      Output:

```
< batch_id_1
```

   Penjelasan:

   Memverifikasi data di mongoshell, apakah data dummy yang digenerate dari bahasa python di visual studio code sudah masuk ke database di mongodb atau belum, lalu membuat index agar dapat mempercepat agregasi

3. Pipeline 1: Persentase Cacat per Batch

```
db.inspeksi.aggregate([
  { $addFields: {
      persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100
] }
  }},
  { $sort: { persen_cacat: -1 } },
  {$limit :10} // lihat 10 teratas saja
])
```

Output:

```
< {
    _id: ObjectId('69f84cd5e371fd1bacdc5e15'),
    batch_id: 'B1767',
    mesin: 'M01',
    tanggal: 2026-06-15T22:35:00.000Z,
    shift: 3,
    inspektor: 'Citra',
    jumlah_diperiksa: 211,
    cacat_ditemukan: 21,
    jenis_cacat: [
      'retak',
      'pori',
      'gores'
    ],
    persen_cacat: 9.95260663507109
  }
  {
    _id: ObjectId('69f84cd5e371fd1bacdc5ee3'),
    batch_id: 'B1051',
    mesin: 'M06',
    tanggal: 2026-05-10T04:04:00.000Z,
    shift: 1,
    inspektor: 'Eka',
    jumlah_diperiksa: 405,
    cacat_ditemukan: 40,
    jenis_cacat: [
      'dimensi',
      'retak',
      'pori'
    ],
    persen_cacat: 9.876543209876543
  }
  {
    _id: ObjectId('69f84cd5e371fd1bacdc6275'),
    batch_id: 'B1539',
    mesin: 'M07',
    tanggal: 2026-05-02T16:54:00.000Z,
    shift: 2,
    inspektor: 'Dian',
    jumlah_diperiksa: 345,
    cacat_ditemukan: 34,
    jenis_cacat: [
      'warna'
    ],
```

Penjelasan:

Mengolah data perbatch, menghitung persentase cacat dan melihat 10 data saja dari atas.

4. Pipeline 2: Gabung dengan Target dan Tentukan Status Batch

```
db.inspeksi.aggregate([
  { $lookup: {
      from: "target_kualitas",
```

```
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100
] },
    target_maks: "$target_info.target_maks_cacat"
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_maks"] }, then: "NOT OK", else:
"OK" } }
  }},
  { $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: { $subtract: ["$persen_cacat", "$target_maks"] }
  }},
  { $sort: { selisih: -1 } },
  { $limit: 10}
])
```

Output:

```
{
  batch_id: 'B1767',
  mesin: 'M01',
  persen_cacat: 9.95260663507109,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.95260663507109
}
{
  batch_id: 'B1136',
  mesin: 'M07',
  persen_cacat: 9.75609756097561,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.75609756097561
}
{
  batch_id: 'B0085',
  mesin: 'M01',
  persen_cacat: 9.615384615384617,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.615384615384617
}
{
  batch_id: 'B1926',
  mesin: 'M09',
  persen_cacat: 9.6,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.6
}
{
  batch_id: 'B0830',
  mesin: 'M05',
  persen_cacat: 9.409190371991247,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.409190371991247
}
```

Penjelasan:

Menggabungkan data inspeksi dengan target_kualitas berdasarkan batch_id, lalu persentase cacat dihitung dan dibandingkan dengar target_maks_cacat. Jika persen_cacat > target_maks_cacat → status "NOT OK", sebaliknya "OK" lalu menampilkan selisihnya.

5. Pipeline 3: Jumlah Batch NOT OK per Mesin per Minggu

```
db.inspeksi.aggregate([
  { $lookup: {
     from: "target_kualitas",
     localField: "batch_id",
     foreignField: "batch_id",
     as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
     persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100
] }
  }},
  { $addFields: {
     status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then:
"NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
```

```
  { $group: {
      _id: {
        mesin: "$mesin",
        minggu: { $week: "$tanggal" },
        tahun: { $year: "$tanggal" }
      },
      jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

Output:

```
< {
    _id: {
      mesin: 'M01',
      minggu: 13,
      tahun: 2026
    },
    jumlah_NOT_OK: 1
  }
  {
    _id: {
      mesin: 'M02',
      minggu: 13,
      tahun: 2026
    },
    jumlah_NOT_OK: 1
  }
  {
    _id: {
      mesin: 'M05',
      minggu: 13,
      tahun: 2026
    },
    jumlah_NOT_OK: 1
  }
  {
    _id: {
      mesin: 'M06',
      minggu: 13,
      tahun: 2026
    },
    jumlah_NOT_OK: 2
  }
  {
    _id: {
      mesin: 'M07',
      minggu: 13,
      tahun: 2026
    },
    jumlah_NOT_OK: 1
  }
```

Penjelasan:

Mengekstrak minggu dari tanggal menggunakan $week, lalu filter hanya batch dengan status NOT OK, kemudian group per mesin dan minggu.

6. Pipeline 4: Top 3 Mesin dengan Jumlah NOT OK Terbanyak (dalam satu bulan tertentu)

```
db.inspeksi.aggregate([
  { $match: {
      tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
```

```
  }},
  { $lookup: {
     from: "target_kualitas",
     localField: "batch_id",
     foreignField: "batch_id",
     as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
     persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100
] }
  }},
  { $addFields: {
     status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then:
"NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
     _id: "$mesin",
     total_NOT_OK: { $sum: 1 }
  }},
  { $sort: { total_NOT_OK: -1 } },
  { $limit: 3 }
])
```

Output:

```
< {
    _id: 'M07',
    total_NOT_OK: 22
  }
  {
    _id: 'M03',
    total_NOT_OK: 20
  }
  {
    _id: 'M09',
    total_NOT_OK: 16
  }
```

Penjelasan:

Menggunakan $match di awal untuk filter bulan, lalu menggunakan join dan penentuan status, lalu group per mesin dan menghitung total NOT OK, diurutkan dan limit 3.

7. Pipeline 5 (Bonus/Opsional): Ringkasan Bulanan dengan $facet

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
```

```
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100
] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan",
"$jumlah_diperiksa"] }, 100 ] }, "$target_info.target_maks_cacat" ] }, then: "NOT OK", else:
"OK" } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  }}
])
```

Output:

```
< {
    total_batch: [
      {
        count: 499
      }
    ],
    rata_cacat: [
      {
        _id: null,
        avg_cacat: 5.064083330063629
      }
    ],
    mesin_terburuk: [
      {
        _id: 'M02',
        jumlah: 21
      },
      {
        _id: 'M05',
        jumlah: 19
      },
      {
        _id: 'M10',
        jumlah: 18
      }
    ]
  }
```


Penjelasan:

Menggunakan $facet untuk memperoleh tiga informasi dalam satu query yaitu: Total produksi (jumlah batch) per bulan, rata-rata persentase cacat per bulan, dan mesin dengan jumlah NOT OK terbanyak per bulan.

Analisis dan Kesimpulan

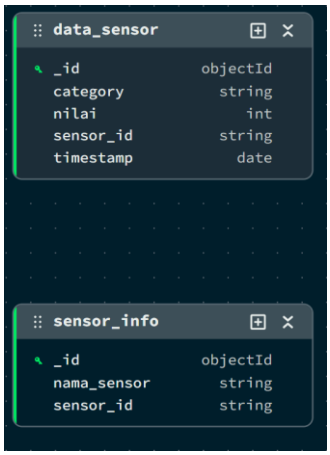
Berdasarkan pengolahan data yang dilakukan, sistem ini sudah mampu menggambarkan kondisi kualitas produksi secara cukup jelas. Dari hasil perhitungan persentase cacat, terlihat bahwa masih ada beberapa batch yang memiliki tingkat cacat cukup tinggi dibandingkan yang lain. Hal ini menunjukkan bahwa proses produksi belum sepenuhnya stabil dan masih terdapat variasi kualitas antar batch.

Setelah data inspeksi digabungkan dengan target kualitas, terlihat bahwa tidak semua batch mampu memenuhi standar yang telah ditentukan. Beberapa batch masuk kategori NOT OK, artinya persentase cacatnya melebihi batas maksimal. Selain itu, jika dilihat berdasarkan mesin dan waktu, terdapat pola di mana mesin tertentu lebih sering menghasilkan batch dengan status NOT OK, terutama pada periode tertentu. Ini mengindikasikan kemungkinan adanya masalah pada performa mesin atau kondisi operasional.

Analisis per minggu dan per bulan juga memperlihatkan bahwa jumlah batch bermasalah tidak merata, melainkan cenderung muncul pada waktu-waktu tertentu. Hal ini bisa disebabkan oleh faktor seperti beban produksi, kondisi mesin, atau bahkan faktor manusia seperti operator dan shift kerja.

C. Latihan Mandiri

Desain Data:



1. Membuat Database

```
use latihan5
```

Output:

```
switched to db latihan5
```

Penjelasan:

Membuat database latihan5

2. Membuat Data Dummy

```
db.data_sensor.drop()

let docs = []

let sensorList = []
for (let i = 1; i <= 20; i++) {
  sensorList.push("S" + i.toString().padStart(2, '0'))
}

// mulai dari 30 hari terakhir
let start = new Date()
start.setDate(start.getDate() - 30)

for (let i = 0; i < 5000; i++) {
  let waktu = new Date(start.getTime() + Math.random() * 30 * 24 * 60 * 60 * 1000)

  docs.push({
    sensor_id: sensorList[Math.floor(Math.random() * sensorList.length)],
    nilai: Math.floor(Math.random() * 101), // 0 - 100
    timestamp: waktu,
    category: ["A", "B", "C"][Math.floor(Math.random() * 3)]
  })
```

```
  // insert per 500 biar ringan
  if (docs.length === 500) {
    db.data_sensor.insertMany(docs)
    docs = []
  }
}

// sisa data
if (docs.length > 0) {
  db.data_sensor.insertMany(docs)
}
```

Output:

```
< []
```

Penjelasan:

Membuat Data dummy sebanyak 5000 dokumen

3. Menampilkan rata-rata nilai per sensor_id pada 7 hari terakhir

```
db.data_sensor.aggregate([
  {
    $match: {
      timestamp: {
        $gte: new Date(Date.now() - 7*24*60*60*1000)
      }
    }
  },
  {
    $group: {
      _id: "$sensor_id",
      rata_nilai: { $avg: "$nilai" }
    }
  }
])
```

Output:

```
< {
  _id: 'S20',
  rata_nilai: 47.127272727272725
}
{
  _id: 'S08',
  rata_nilai: 52.46666666666667
}
{
  _id: 'S10',
  rata_nilai: 51
}
{
  _id: 'S04',
  rata_nilai: 58.96666666666667
}
{
  _id: 'S16',
  rata_nilai: 55.24074074074074
}
{
  _id: 'S02',
  rata_nilai: 51.8
}
{
  _id: 'S03',
  rata_nilai: 53.96875
}
{
  _id: 'S13',
  rata_nilai: 57.44642857142857
}
{
  _id: 'S06',
  rata_nilai: 48.80701754385965
}
{
  _id: 'S05',
  rata_nilai: 50.868852459016395
}
```

Penjelasan:

- $match memfilter data hanya untuk 7 hari terakhir agar analisis lebih relevan.
- $group mengelompokkan berdasarkan sensor_id dan menghitung rata-rata nilai

4. Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan $group dengan $push atau $sum kondisional).

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      count_high: {
        $sum: {
          $cond: [{ $gt: ["$nilai", 90] }, 1, 0]
        }
      }
    }
  },
  {
    $match: { count_high: { $gte: 3 } }
  }
])
```

Output:

```
< {
    _id: 'S13',
    count_high: 23
  }
  {
    _id: 'S09',
    count_high: 23
  }
  {
    _id: 'S12',
    count_high: 34
  }
  {
    _id: 'S05',
    count_high: 29
  }
  {
    _id: 'S20',
    count_high: 21
  }
  {
    _id: 'S11',
    count_high: 27
  }
  {
    _id: 'S18',
    count_high: 20
  }
  {
    _id: 'S19',
    count_high: 16
  }
  {
    _id: 'S04',
    count_high: 30
  }
  {
    _id: 'S08',
    count_high: 34
  }
```

Penjelasan:

- $group menghitung jumlah kemunculan nilai di atas 90 per sensor menggunakan kondisi
- $match memfilter hanya sensor yang memiliki nilai tinggi minimal 3 kali

5. Menambahkan field jam dan menghitung nilai rata rata per jam per sensor.

```
db.data_sensor.aggregate([
  {
    $addFields: {
      jam: { $hour: "$timestamp" }
    }
  },
  {
    $group: {
      _id: { sensor: "$sensor_id", jam: "$jam" },
      rata_nilai: { $avg: "$nilai" }
    }
  }
])
```

Output:

```
< {
    _id: {
      sensor: 'S03',
      jam: 12
    },
    rata_nilai: 43
  }
  {
    _id: {
      sensor: 'S01',
      jam: 0
    },
    rata_nilai: 41.8
  }
  {
    _id: {
      sensor: 'S08',
      jam: 10
    },
    rata_nilai: 52.8
  }
  {
    _id: {
      sensor: 'S06',
      jam: 2
    },
    rata_nilai: 50.333333333333336
  }
  {
    _id: {
      sensor: 'S05',
      jam: 23
    },
    rata_nilai: 49.4
  }
  {
    _id: {
      sensor: 'S17',
      jam: 8
    },
    rata_nilai: 38.63636363636363
  }
```

Penjelasan:

- $addFields menambahkan field baru jam dari timestamp
- $group mengelompokkan data berdasarkan sensor dan jam, lalu menghitung rata-rata nilai

6. Buat koleksi sensor_info lalu join untuk menampilkan nama sensor dalam hasil.

```
db.data_sensor.aggregate([
  {
   $group: {
     _id: "$sensor_id",
     rata_nilai: { $avg: "$nilai" }
   }
  },
  {
   $lookup: {
     from: "sensor_info",
     localField: "_id",
     foreignField: "sensor_id",
```

```
      as: "info"
    }
  },
  {
    $unwind: "$info"
  }
])
```

Output:

```
< {
    _id: 'S01',
    rata_nilai: 46.84304932735426,
    info: {
      _id: ObjectId('69f8a092ba9740cae430fd50'),
      sensor_id: 'S01',
      nama_sensor: 'Sensor A'
    }
  }
  {
    _id: 'S02',
    rata_nilai: 50.08366533864542,
    info: {
      _id: ObjectId('69f8a092ba9740cae430fd51'),
      sensor_id: 'S02',
      nama_sensor: 'Sensor B'
    }
  }
```

Penjelasan:

- $group menghitung rata-rata nilai per sensor
- $lookup menggabungkan data dengan koleksi sensor_info berdasarkan sensor_id
- $unwind mengubah hasil join dari array menjadi objek
- $project menampilkan field yang dibutuhkan agar hasil lebih rapi

Analisis dan Kesimpulan

Berdasarkan hasil pengolahan data sensor menggunakan aggregation pipeline di MongoDB, dapat dilihat bahwa setiap sensor memiliki pola nilai yang berbeda-beda dalam periode waktu tertentu. Dari perhitungan rata-rata nilai pada 7 hari terakhir, terlihat bahwa beberapa sensor cenderung stabil, sementara yang lain menunjukkan fluktuasi yang cukup besar.

Selain itu, dari analisis nilai di atas 90, ditemukan bahwa hanya beberapa sensor yang sering mencapai nilai tinggi secara berulang. Hal ini bisa menjadi indikasi bahwa sensor tersebut bekerja pada kondisi ekstrem atau terdapat potensi anomali yang perlu diperhatikan lebih lanjut.

Pada analisis berdasarkan waktu (per jam), terlihat adanya variasi nilai sensor pada jam-jam tertentu. Hal ini menunjukkan bahwa kondisi lingkungan atau proses yang dipantau oleh sensor dapat berubah tergantung waktu, sehingga penting untuk melakukan monitoring secara berkala.

Penggunaan $lookup untuk menggabungkan data dengan koleksi sensor_info memberikan informasi tambahan berupa nama sensor, sehingga hasil analisis menjadi lebih mudah dipahami dan lebih informatif.

PENUTUP

Dari hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan aggregation pipeline pada **MongoDB** sangat membantu dalam proses analisis data, mulai dari filtering, pengelompokan, hingga penggabungan data antar koleksi.

Pipeline yang dibuat mampu memberikan gambaran mengenai pola nilai sensor, mendeteksi nilai ekstrem, serta melihat distribusi data berdasarkan waktu. Selain itu, penggunaan $lookup juga membuat hasil analisis menjadi lebih informatif karena dapat menampilkan data tambahan dari koleksi lain.

Dengan memahami konsep dan implementasi aggregation pipeline, proses pengolahan data dapat dilakukan dengan lebih efektif dan efisien. Diharapkan kemampuan ini dapat digunakan dalam pengembangan sistem monitoring maupun analisis data yang lebih kompleks di masa mendatang.